SME_R — Ejemplo de uso

Contents

Contenido . . . 2
Durante la creación del paquete . . . 2
1. Instalación e importación . . . 2
Comprobación de la instalación . . . 2
2. Dataset . . . 3
Carga del dataset . . . 3
Exploración inicial . . . 4
3. Discretización de variables . . . 4
3.1. Discretización automática . . . 5
3.2. Discretización por igual anchura . . . 6
3.3. Discretización por igual frecuencia . . . 7
3.4. Especificar directamente la frecuencia . . . 8
4. Cálculo de métricas . . . 9
5. Normalización y estandarización . . . 11
6. Filtrado de variables . . . 12
7. Correlación e información mutua . . . 14
8. Visualización de resultados . . . 15
Conclusión . . . 21


```
install.packages("devtools")
install.packages("ggplot2")
```

```
library(ggplot2)
```

Este documento muestra cómo utilizar **SME_R**, una librería orientada al preprocesamiento y análisis de datasets.

A lo largo del documento utilizaremos un dataset real de vehículos para mostrar, de forma progresiva, las principales funcionalidades del paquete.

Contenido

1. Instalación e importación del paquete
2. Carga y exploración del dataset
3. Discretización de variables
4. Cálculo de métricas
5. Normalización y estandarización
6. Filtrado de variables
7. Correlación e información mutua
8. Visualización de resultados

Durante la creación del paquete

Se ha intentado mantener un estilo de API publica (ej. `discretizar()`) que llama a funciones internas (`.discretizar_frecuencia`) que el usuario no puede ver, haciendo un 'router' interno según el caso.

También se ha hecho hincapié en un sistema de impresión de información a distintos niveles (`verbose`) y se ha intentado gestionar todos los errores, warnings… mediante el uso de `verbose` y `raise_error`.

1. Instalación e importación

En primer lugar cargamos la librería.

```
devtools::load_all("C:/Users/Burbu/Desktop/SME_Python/R/src/SME_R") # Durante la creacion y debuugging
```

```
devtools::document("../R/src/SME_R") #Para documentacion automatica
```

```
#detach("package:SMER", unload = TRUE)
devtools::install_github(
  "EnZaFe/SME_paquetes",
  subdir = "R/src/SME_R",
  force = TRUE
)
```

```
library(SMER)
```

Comprobación de la instalación

Comprobamos que el paquete se puede cargar correctamente y mostramos la versión instalada.

```
packageVersion("SMER")
```

```
## [1] '0.1.0'
```

```
#setwd("C:/Users/Burbu/Desktop/SME_Python/R/src/SME_R")
#roxygen2::roxygenise(clean = TRUE)
```

2. Dataset

Para probar las funcionalidades de la librería utilizaremos el **Vehicle Dataset from CarDekho** además de ciertos datasets clásicos (iris, wine, breast_cancer).

El dataset contiene información sobre vehículos de segunda mano e incluye tanto variables numéricas como categóricas.

Entre las variables disponibles encontramos:

- `name`: nombre del vehículo
- `year`: año del vehículo.
- `selling_price`: precio de venta.
- `km_driven`: kilómetros recorridos.
- `fuel`: tipo de combustible.
- `seller_type`: tipo de vendedor.
- `transmission`: tipo de transmisión.
- `owner`: número de propietarios anteriores.

Carga del dataset

Si el dataset no está disponible como `car_data.csv`.

```
download.file(
  "https://www.kaggle.com/api/v1/datasets/download/nehalbirla/vehicle-dataset-from-cardekho",
  destfile = "vehicle-dataset.zip",
  mode = "wb"
)
unzip("vehicle-dataset.zip")
list.files()
```

```
## [1] "car data.csv"
## [2] "CAR DETAILS FROM CAR DEKHO.csv"
## [3] "Car details v3.csv"
## [4] "car details v4.csv"
## [5] "demo_python.ipynb"
## [6] "demo_r_1.Rmd"
## [7] "figures"
## [8] "Libreria SME_Python - Ejemplo de uso.ipynb"
## [9] "vehicle-dataset.zip"
```

```
df <- read.csv("car data.csv")
```

En caso de que ya tengamos un dataset descargado o queramos hacer pruebas, podemos utilizar la función `load_dataset`, con la que podemos convertir automáticamente a `data.frame` cualquier archivo de excel o texto. Además, nos permite cargar directamente 3 datasets clásicos: iris, wine y breast_cancer.

```
df <- cargar_dataset("CAR DETAILS FROM CAR DEKHO.csv")
```

```
## Cargando texto delimitado desde 'CAR DETAILS FROM CAR DEKHO.csv'...
```

```
## Separador detectado: ','
```

```
## Cargado: 4340 filas x 8 columnas.
```

```
class(df)
```

```
## [1] "data.frame"
```

```
df_iris <- cargar_dataset("iris")
```

```
## Cargando dataset 'iris'...
```

```
## Cargado: 150 filas x 5 columnas.
```

```
class(df_iris)
```

```
## [1] "data.frame"
```

Exploración inicial

Antes de aplicar las funciones de `SME_R`, comprobamos las dimensiones del dataset, sus columnas y los tipos de datos.

```
dim(df)
```

```
## [1] 4340    8
```

```
str(df)
```

```
## 'data.frame':    4340 obs. of  8 variables:
##  $ name         : chr  "Maruti 800 AC" "Maruti Wagon R LXI Minor" "Hyundai Verna 1.6 SX" "Datsun Redi
##  $ year         : num  2007 2007 2012 2017 2014 ...
##  $ selling_price: num  60000 135000 600000 250000 450000 140000 550000 240000 850000 365000 ...
##  $ km_driven    : num  70000 50000 100000 46000 141000 125000 25000 60000 25000 78000 ...
##  $ fuel         : chr  "Petrol" "Petrol" "Diesel" "Petrol" ...
##  $ seller_type  : chr  "Individual" "Individual" "Individual" "Individual" ...
##  $ transmission : chr  "Manual" "Manual" "Manual" "Manual" ...
##  $ owner        : chr  "First Owner" "First Owner" "First Owner" "First Owner" ...
```

3. Discretización de variables

La discretización permite transformar variables numéricas continuas en intervalos.

`SME_R` proporciona dos métodos:

- **Anchura (`anchura`)**: divide el rango de valores en intervalos de igual amplitud.
- **Frecuencia (`frecuencia`)**: construye intervalos intentando mantener una cantidad similar de observaciones en cada uno.

La función también permite seleccionar qué columnas discretizar y controlar el número de intervalos o la frecuencia deseada.

3.1. Discretización automática

Si no especificamos las columnas, la función identifica las variables numéricas susceptibles de ser discretizadas. Si tampoco indicamos el número de intervalos, la función utiliza su criterio predeterminado para determinarlo.

```
df_discret <- discretizar(df)
```

```
## Iniciando discretizacion...

## Warning: No se ha especificado n_intervalos. Se utilizarán 2170 intervalos (50%
## del número de casos).
## Warning: No se ha especificado n_intervalos. Se utilizarán 2170 intervalos (50%
## del número de casos).
## Warning: No se ha especificado n_intervalos. Se utilizarán 2170 intervalos (50%
## del número de casos).
```

```
head(df_discret)
```

```
##                       name             year      selling_price
## 1            Maruti 800 AC 2006.99, 2007.01 56829.49, 60921.66
## 2 Maruti Wagon R LXI Minor 2006.99, 2007.01 134580.6, 138672.8
## 3     Hyundai Verna 1.6 SX 2011.99, 2012.00 596995.4, 601087.6
## 4   Datsun RediGO T Option 2016.99, 2017.01 249161.3, 253253.5
## 5    Honda Amaze VX i-DTEC 2013.99, 2014.00 449677.4, 453769.6
## 6     Maruti Alto LX BSIII 2006.99, 2007.01 138672.8, 142765.0
##             km_driven   fuel seller_type transmission        owner
## 1  69881.38, 70253.08 Petrol  Individual       Manual  First Owner
## 2  49809.36, 50181.06 Petrol  Individual       Manual  First Owner
## 3 99989.42, 100361.12 Diesel  Individual       Manual  First Owner
## 4  45720.61, 46092.31 Petrol  Individual       Manual  First Owner
## 5  140876.9, 141248.6 Diesel  Individual       Manual Second Owner
## 6  124893.6, 125265.3 Petrol  Individual       Manual  First Owner
```

Como no podemos ver los numeros directamente vamos a mostrar las primeras filas del dataset discretizado.

```
tabla_con_vectores_visual <- function(df) {

  cols_problematicas <- names(df)[sapply(df, function(x) {
    any(vapply(x, length, integer(1)) > 1)
  })]

  df_visual <- df

  # Convertir los vectores internos a texto
  df_visual[cols_problematicas] <- lapply(
    df_visual[cols_problematicas],
    function(x) {
      vapply(
        x,
        function(y) paste(y, collapse = ", "),
        character(1)
```

```
      )
    }
  )

  df_visual
}


head(tabla_con_vectores_visual(df_discret))
```

```
##                       name             year        selling_price
## 1            Maruti 800 AC 2006.99, 2007.01   56829.49, 60921.66
## 2 Maruti Wagon R LXI Minor 2006.99, 2007.01 134580.65, 138672.81
## 3     Hyundai Verna 1.6 SX    2011.99, 2012 596995.39, 601087.56
## 4   Datsun RediGO T Option 2016.99, 2017.01 249161.29, 253253.46
## 5    Honda Amaze VX i-DTEC    2013.99, 2014 449677.42, 453769.59
## 6     Maruti Alto LX BSIII 2006.99, 2007.01 138672.81, 142764.98
##              km_driven   fuel seller_type transmission        owner
## 1   69881.38, 70253.08 Petrol  Individual       Manual  First Owner
## 2   49809.36, 50181.06 Petrol  Individual       Manual  First Owner
## 3  99989.42, 100361.12 Diesel  Individual       Manual  First Owner
## 4   45720.61, 46092.31 Petrol  Individual       Manual  First Owner
## 5 140876.87, 141248.58 Diesel  Individual       Manual Second Owner
## 6  124893.59, 125265.3 Petrol  Individual       Manual  First Owner
```

3.2. Discretización por igual anchura

Podemos seleccionar una variable concreta y especificar el número de intervalos que queremos utilizar.

En este caso dividiremos `km_driven` en 10 intervalos de igual anchura.

```
df_discret <- discretizar(
  df,
  columnas = list("km_driven"),
  metodo = "anchura",
  n_intervalos = 10,
  verbose = 2
)
```

```
## Iniciando discretizacion...

## Parametros elegidos:
## metodo: anchura,
## n_intervalos: 10,
## frecuenca: ,
## columnas: km_driven

## Número de casos: 4340

## Mínimo: 1

## Máximo: 806599
```

```
## Anchura de los intervalos: 80659.8

## Límites de los intervalos: 1, 80660.8, 161320.6, 241980.4, 322640.2, 403300, 483959.8, 564619.6, 645
```

```
head(tabla_con_vectores_visual(df_discret))
```

```
##                       name year selling_price         km_driven   fuel
## 1            Maruti 800 AC 2007         60000        1, 80660.8 Petrol
## 2 Maruti Wagon R LXI Minor 2007        135000        1, 80660.8 Petrol
## 3     Hyundai Verna 1.6 SX 2012        600000 80660.8, 161320.6 Diesel
## 4   Datsun RediGO T Option 2017        250000        1, 80660.8 Petrol
## 5    Honda Amaze VX i-DTEC 2014        450000 80660.8, 161320.6 Diesel
## 6     Maruti Alto LX BSIII 2007        140000 80660.8, 161320.6 Petrol
##   seller_type transmission        owner
## 1  Individual       Manual  First Owner
## 2  Individual       Manual  First Owner
## 3  Individual       Manual  First Owner
## 4  Individual       Manual  First Owner
## 5  Individual       Manual Second Owner
## 6  Individual       Manual  First Owner
```

3.3. Discretización por igual frecuencia

También podemos utilizar el método de igual frecuencia.

En este caso los intervalos se construyen de manera que contengan, aproximadamente, el mismo número de observaciones.

```
df_discret <- discretizar(
  df,
  columnas = list("km_driven"),
  metodo = "frecuencia",
  n_intervalos = 10,
  verbose = 2
)
```

```
## Iniciando discretizacion...

## Parametros elegidos:
## metodo: frecuencia,
## n_intervalos: 10,
## frecuenca: ,
## columnas: km_driven

## Número de casos: 4340

## Frecuencia base calculada: 434

## Elementos sobrantes: 0

## Primeros valores ordenados: 1, 101, 101, 101, 1000, 1000, 1000, 1000, 1000, 1000
```

```r
head(tabla_con_vectores_visual(df_discret))
```

```
##                       name year selling_price      km_driven   fuel seller_type
## 1            Maruti 800 AC 2007         60000   60000, 70000 Petrol  Individual
## 2 Maruti Wagon R LXI Minor 2007        135000   40000, 50000 Petrol  Individual
## 3     Hyundai Verna 1.6 SX 2012        600000   80000, 1e+05 Diesel  Individual
## 4   Datsun RediGO T Option 2017        250000   40000, 50000 Petrol  Individual
## 5    Honda Amaze VX i-DTEC 2014        450000 120000, 806599 Diesel  Individual
## 6     Maruti Alto LX BSIII 2007        140000 120000, 806599 Petrol  Individual
##   transmission        owner
## 1       Manual  First Owner
## 2       Manual  First Owner
## 3       Manual  First Owner
## 4       Manual  First Owner
## 5       Manual Second Owner
## 6       Manual  First Owner
```

3.4. Especificar directamente la frecuencia

Otra posibilidad es indicar directamente cuántas observaciones queremos aproximadamente en cada intervalo.

La función se encarga de calcular automáticamente los límites necesarios.

```r
df_discret <- discretizar(
  df,
  columnas = list("km_driven"),
  metodo = "frecuencia",
  frecuencia = 10,
  verbose = 2
)
```

```
## Iniciando discretizacion...

## Parametros elegidos:
## metodo: frecuencia,
## n_intervalos: ,
## frecuenca: 10,
## columnas: km_driven

## Número de casos: 4340

## Elementos sobrantes: 0

## Primeros valores ordenados: 1, 101, 101, 101, 1000, 1000, 1000, 1000, 1000, 1000
```

```r
head(tabla_con_vectores_visual(df_discret))
```

```
##                       name year selling_price      km_driven   fuel seller_type
## 1            Maruti 800 AC 2007         60000   68745, 70000 Petrol  Individual
## 2 Maruti Wagon R LXI Minor 2007        135000   49824, 50000 Petrol  Individual
## 3     Hyundai Verna 1.6 SX 2012        600000   1e+05, 1e+05 Diesel  Individual
```

```
## 4   Datsun RediGO T Option 2017        250000   46000, 46000 Petrol  Individual
## 5    Honda Amaze VX i-DTEC 2014        450000 140000, 144000 Diesel  Individual
## 6     Maruti Alto LX BSIII 2007        140000 124000, 125000 Petrol  Individual
##   transmission        owner
## 1       Manual  First Owner
## 2       Manual  First Owner
## 3       Manual  First Owner
## 4       Manual  First Owner
## 5       Manual Second Owner
## 6       Manual  First Owner
```

4. Cálculo de métricas

Una vez que hemos visto cómo transformar las variables, podemos calcular diferentes métricas para estudiar sus características.

Podemos calcular:

- **Entropía** para variables discretas.
- **Varianza** para variables numéricas.
- **AUC** cuando disponemos y asignamos una variable objetivo binaria, es decir, 2 clases.

Estas métricas pueden utilizarse posteriormente para seleccionar o filtrar variables.

```
metricas <- calcular_metricas(df, verbose = 1)
```

```
## Iniciando cálculo de métricas...

## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
```

```
metricas
```

```
## $name
## $name$entropia
## [1] 9.805485
##
##
## $year
## $year$varianza
## [1] 17.76503
##
##
## $selling_price
## $selling_price$varianza
## [1] 334641515977
##
##
## $km_driven
## $km_driven$varianza
## [1] 2175170962
```

```
##
##
## $fuel
## $fuel$entropia
## [1] 1.111502
##
##
## $seller_type
## $seller_type$entropia
## [1] 0.9280578
##
##
## $transmission
## $transmission$entropia
## [1] 0.4791385
##
##
## $owner
## $owner$entropia
## [1] 1.311691
```

También podemos calcular las métricas sobre un conjunto concreto de variables y asignando clases para el cálculo de AUC.

```
metricas <- calcular_metricas(
  df,
  clases = df$Transmission,
  verbose=1
)
```

```
## Iniciando cálculo de métricas...

## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
```

```
metricas
```

```
## $name
## $name$entropia
## [1] 9.805485
##
##
## $year
## $year$varianza
## [1] 17.76503
##
##
## $selling_price
## $selling_price$varianza
## [1] 334641515977
##
##
```

```
## $km_driven
## $km_driven$varianza
## [1] 2175170962
##
##
## $fuel
## $fuel$entropia
## [1] 1.111502
##
##
## $seller_type
## $seller_type$entropia
## [1] 0.9280578
##
##
## $transmission
## $transmission$entropia
## [1] 0.4791385
##
##
## $owner
## $owner$entropia
## [1] 1.311691
```

5. Normalización y estandarización

La librería incluye dos transformaciones habituales para preparar variables numéricas:

- **Normalización**: transforma los valores al rango `[0, 1]`.
- **Estandarización**: transforma los valores para trabajar alrededor de una media de 0 y una desviación estándar de 1.

En ambos casos podemos indicar las variables que queremos transformar.

```
df_normalizado <- normalizar(
  df,
  atributos = list("selling_price", "km_driven")
)
```

```
## Iniciando normalización...
```

```
## Warning: La función .es_variable está obsoleta. Usa .es_continua en su lugar.
```

```
head(df_normalizado)
```

```
##                       name year selling_price  km_driven   fuel seller_type
## 1            Maruti 800 AC 2007   0.004504505 0.08678301 Petrol  Individual
## 2 Maruti Wagon R LXI Minor 2007   0.012950450 0.06198751 Petrol  Individual
## 3     Hyundai Verna 1.6 SX 2012   0.065315315 0.12397626 Diesel  Individual
## 4   Datsun RediGO T Option 2017   0.025900901 0.05702841 Petrol  Individual
## 5    Honda Amaze VX i-DTEC 2014   0.048423423 0.17480703 Diesel  Individual
```

```
## 6     Maruti Alto LX BSIII 2007   0.013513514 0.15497063 Petrol  Individual
##   transmission        owner
## 1       Manual  First Owner
## 2       Manual  First Owner
## 3       Manual  First Owner
## 4       Manual  First Owner
## 5       Manual Second Owner
## 6       Manual  First Owner
```

```
df_estandarizado <- estandarizar(
  df,
  atributos = list("selling_price", "km_driven")
)
```

```
## Iniciando estandarización...
```

```
## Warning: La función .es_variable está obsoleta. Usa .es_continua en su lugar.
```

```
head(df_estandarizado)
```

```
##                       name year selling_price   km_driven   fuel seller_type
## 1            Maruti 800 AC 2007   -0.76765756  0.08112971 Petrol  Individual
## 2 Maruti Wagon R LXI Minor 2007   -0.63802285 -0.34764904 Petrol  Individual
## 3     Hyundai Verna 1.6 SX 2012    0.16571238  0.72429784 Diesel  Individual
## 4   Datsun RediGO T Option 2017   -0.43924962 -0.43340479 Petrol  Individual
## 5    Honda Amaze VX i-DTEC 2014   -0.09355705  1.60329429 Diesel  Individual
## 6     Maruti Alto LX BSIII 2007   -0.62938053  1.26027128 Petrol  Individual
##   transmission        owner
## 1       Manual  First Owner
## 2       Manual  First Owner
## 3       Manual  First Owner
## 4       Manual  First Owner
## 5       Manual Second Owner
## 6       Manual  First Owner
```

6. Filtrado de variables

Una vez calculadas las métricas podemos utilizarlas para seleccionar variables.

La función `filtrar_variables` permite conservar o descartar variables según un umbral determinado.

El criterio utilizado depende del tipo de variable y de la métrica seleccionada.

Si no se le dan los datos, hace los cálculos automáticamente.

```
df_filtrado <- filtrar_variables(
  df,
  entropia = c(0.3, 0.4),
  modo = "dentro",
  verbose = 1
)
```

```
## Iniciando filtrado de variables...
```

```
## Iniciando cálculo de métricas...

## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
```

```
# Variables que han sido eliminadas por el filtro.
eliminadas <- setdiff(colnames(df), colnames(df_filtrado))

cat("Variables eliminadas:", length(eliminadas), "\n")
```

```
## Variables eliminadas: 5
```

```
print(eliminadas)
```

```
## [1] "name"         "fuel"         "seller_type"  "transmission" "owner"
```

Se pueden usar como umbrales: la entropía, la varianza y el AUC. Se recomienda hacer una normalización o estandarización antes de utilizar el filtro.

También tiene tanto modo `'dentro'` como `'fuera'` del rango que se le añada a cada umbral.

Destacar que permite el uso de múltiples umbrales al mismo tiempo, donde se aplica discreto a los discretos y numérico a los numéricos. Aunque varianza y AUC a la vez hacen "and".

```
df_filtrado <- filtrar_variables(
  df_estandarizado,
  # entropia = c(0.3, 0.4),
  varianza = c(0.3, 0.5),
  modo = "dentro"
)
```

```
## Iniciando filtrado de variables...

## Iniciando cálculo de métricas...

## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
```

```
eliminadas <- setdiff(colnames(df), colnames(df_filtrado))

cat("Variables eliminadas:", length(eliminadas), "\n")
```

```
## Variables eliminadas: 3
```

```
print(eliminadas)
```

```
## [1] "year"          "selling_price" "km_driven"
```

De todas maneras se puede hacer por partes; calculando las métricas fuera y eligiendo los atributos que se desee.

```r
cat("----------* CALCULANDO METRICAS *----------\n")
```

```
## ----------* CALCULANDO METRICAS *----------
```

```r
metricas <- calcular_metricas(df_estandarizado)
```

```
## Iniciando cálculo de métricas...

## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
## Warning: No se han proporcionado clases. No se calculará el AUC.
```

```r
cat("----------* FILTRANDO DATASET *----------\n")
```

```
## ----------* FILTRANDO DATASET *----------
```

```r
df_filtrado <- filtrar_variables(
  dataset = df_estandarizado,
  # entropia = c(0.3, 0.4),
  varianza = c(0.3, 0.5),
  modo = "dentro",
  metricas = metricas,
  verbose = 2
)
```

```
## Iniciando filtrado de variables...
```

```r
eliminadas <- setdiff(colnames(df), colnames(df_filtrado))

cat("Variables eliminadas:", length(eliminadas), "\n")
```

```
## Variables eliminadas: 3
```

```r
print(eliminadas)
```

```
## [1] "year"          "selling_price" "km_driven"
```

7. Correlación e información mutua

SME_R permite estudiar diferentes tipos de relación entre variables dependiendo de su naturaleza:

- **Pearson** para variables numéricas.
- **Información mutua** para variables categóricas.
- **Welch** para estudiar la relación entre una variable categórica y una numérica.

La función selecciona el cálculo apropiado en función del tipo de variables.

```r
correlaciones <- calcular_correlacion(df, atributos = list("selling_price","transmission","seller_type"
```

```
## Iniciando cálculo de correlaciones...

## Se analizarán 5 atributos.

## Cálculo de correlaciones finalizado.
```

```r
correlaciones$matriz
```

```
##               selling_price transmission  seller_type   km_driven        owner
## selling_price     1.0000000 2.742672e+02  82.33278128  -0.1922886 1.673873e+02
## transmission    274.2671886 1.000000e+00   0.02015531  90.7968688 4.687782e-03
## seller_type      82.3327813 2.015531e-02   1.00000000 146.9632035 4.982377e-02
## km_driven        -0.1922886 9.079687e+01 146.96320350   1.0000000 4.361370e+02
## owner           167.3873237 4.687782e-03   0.04982377 436.1369915 1.000000e+00
```

```r
# El primer elemento contiene los detalles de cada par de variables.
```

Como vemos, distintos tipos de correlaciones utilizan distintas escalas no comparables y que distorsionan lo que vemos. Por ello recibimos 3 matrices, cada una con en su tipo.

8. Visualización de resultados

Finalmente, la librería incorpora funciones de visualización para representar los resultados de algunos de los análisis anteriores.

Entre ellas encontramos visualizaciones para:

- AUC.
- Correlaciones de Pearson.
- Información mutua.
- Estadísticos de Welch.

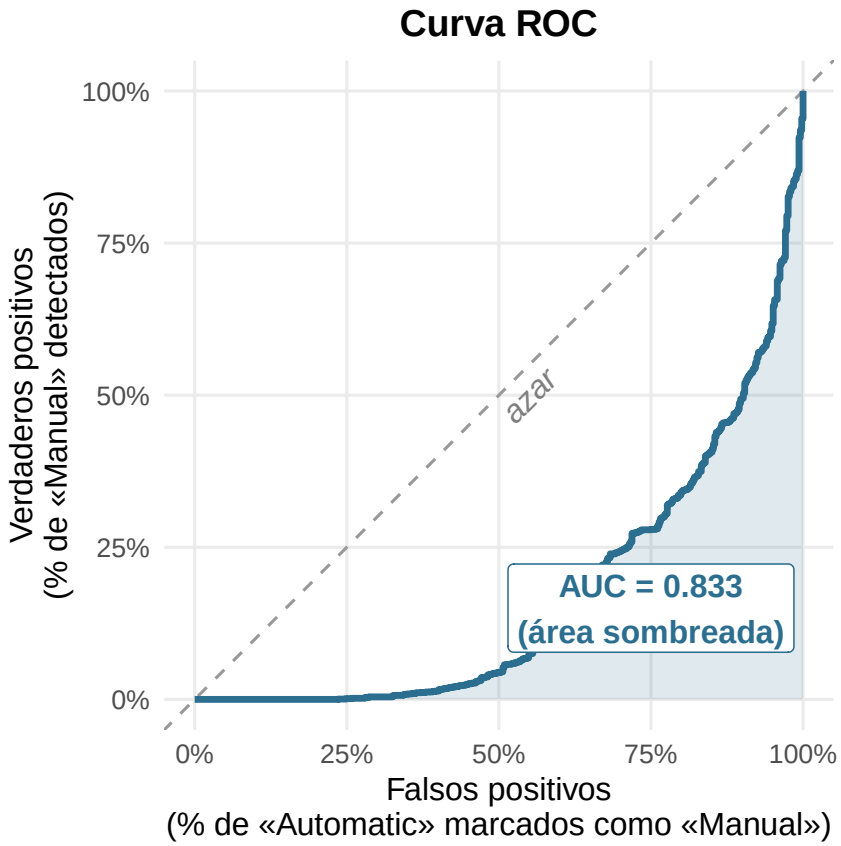
```r
fig1 <- graficar_auc(
  df$selling_price,
  df$transmission
)
```

```
## No se ha proporcionado el resultado del AUC. Se calculará automáticamente.

## Curva ROC · Cómo leerla: cuanto más se acerca la curva a la esquina superior izquierda, mejor separa
## Respuesta: SÍ. AUC = 0.833  0.7: el atributo separa las clases con capacidad suficiente.

## Gráfico de la curva ROC creado correctamente.
```

```r
fig1
```



Vemos un AUC positivo y un SI a la pregunta. mientras que la ilustracion en si misma, enseña una curva muy por debajo del azar. Eso demuestra una clasificacion invertida y por tanto correcta, inversamente.

Intentaremos multiplicar el atributo numerico *-1.

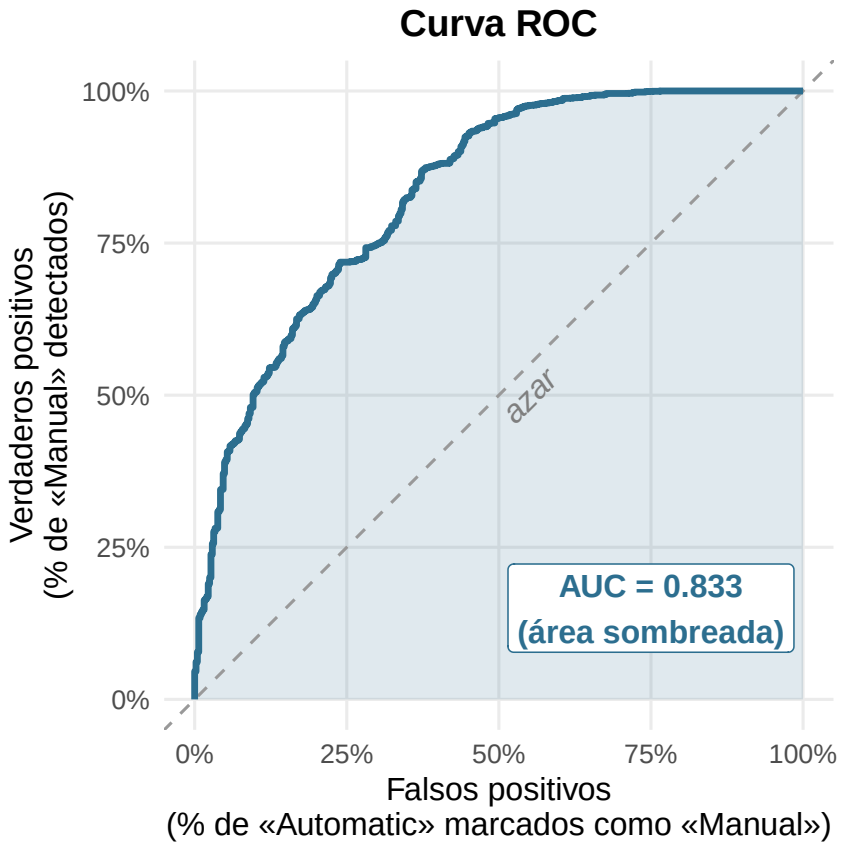
```
fig2 <- graficar_auc(
  df$selling_price*-1,
  df$transmission
  )
```

```
## No se ha proporcionado el resultado del AUC. Se calculará automáticamente.
```

```
## Curva ROC · Cómo leerla: cuanto más se acerca la curva a la esquina superior izquierda, mejor separa
## Respuesta: SÍ. AUC = 0.833  0.7: el atributo separa las clases con capacidad suficiente.
```

```
## Gráfico de la curva ROC creado correctamente.
```

```
fig2
```




```
# Si ya tenemos los resultados podemos pasarlos directamente a la función.
fig1 <- graficar_pearson(
  df$km_driven,
  df$selling_price,
)
```

```
## No se ha proporcionado el resultado de Pearson. Se calculará automáticamente.

## Pearson · Cómo leerlo: cada punto es una observación; la recta es el ajuste lineal y la banda, ±1 des
## Respuesta: NO porque |r| < 0.3: relación débil y negativa; r² = 0.037.

## Gráfico pearson creado correctamente.
```

```
resultado = calcular_correlacion(
    df,
    atributos=list("km_driven", "selling_price")
)
```

```
## Iniciando cálculo de correlaciones...

## Se analizarán 2 atributos.

## Cálculo de correlaciones finalizado.
```

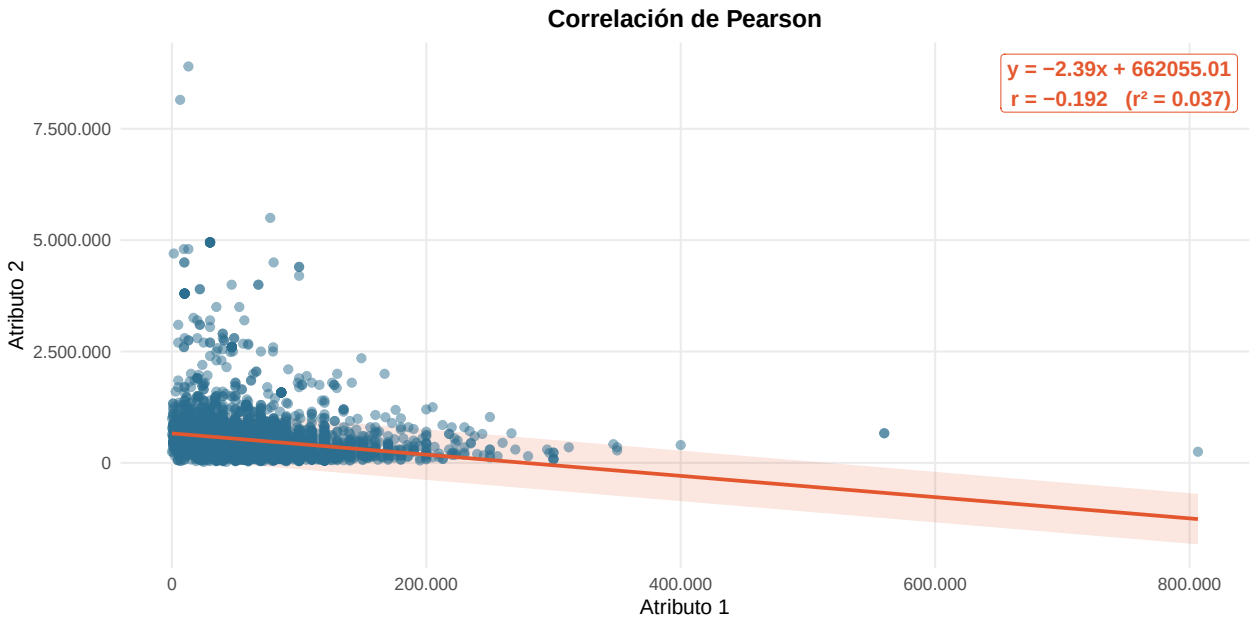
```
fig2 <- graficar_pearson(
    df$km_driven,
    df$selling_price,
    resultado = resultado$detalles[["km_driven|selling_price"]]
)
```

```
## Pearson · Cómo leerlo: cada punto es una observación; la recta es el ajuste lineal y la banda, ±1 des
## Respuesta: NO porque |r| < 0.3: relación débil y negativa; r² = 0.037.
```

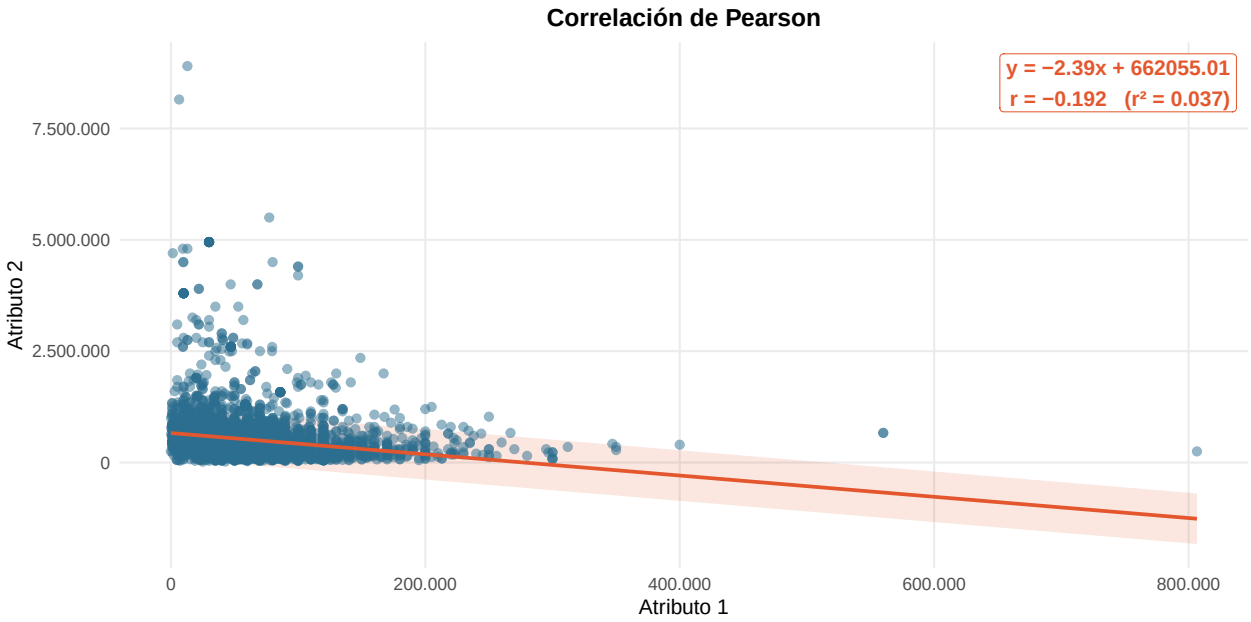
```
## Gráfico pearson creado correctamente.
```

Veamos los dos gráficos uno al lado del otro...

```
fig1
```




```
fig2
```



Vemos como aunque a priori diriamos que más kilometros significa mas barato, hay suficiente ruido de gamas distintas... que no se puede y mirando misma gama o mismo tipo de coche si existiria la relacion.

```
fig <- graficar_informacion_mutua(
  df$seller_type,
  df$owner
)
```

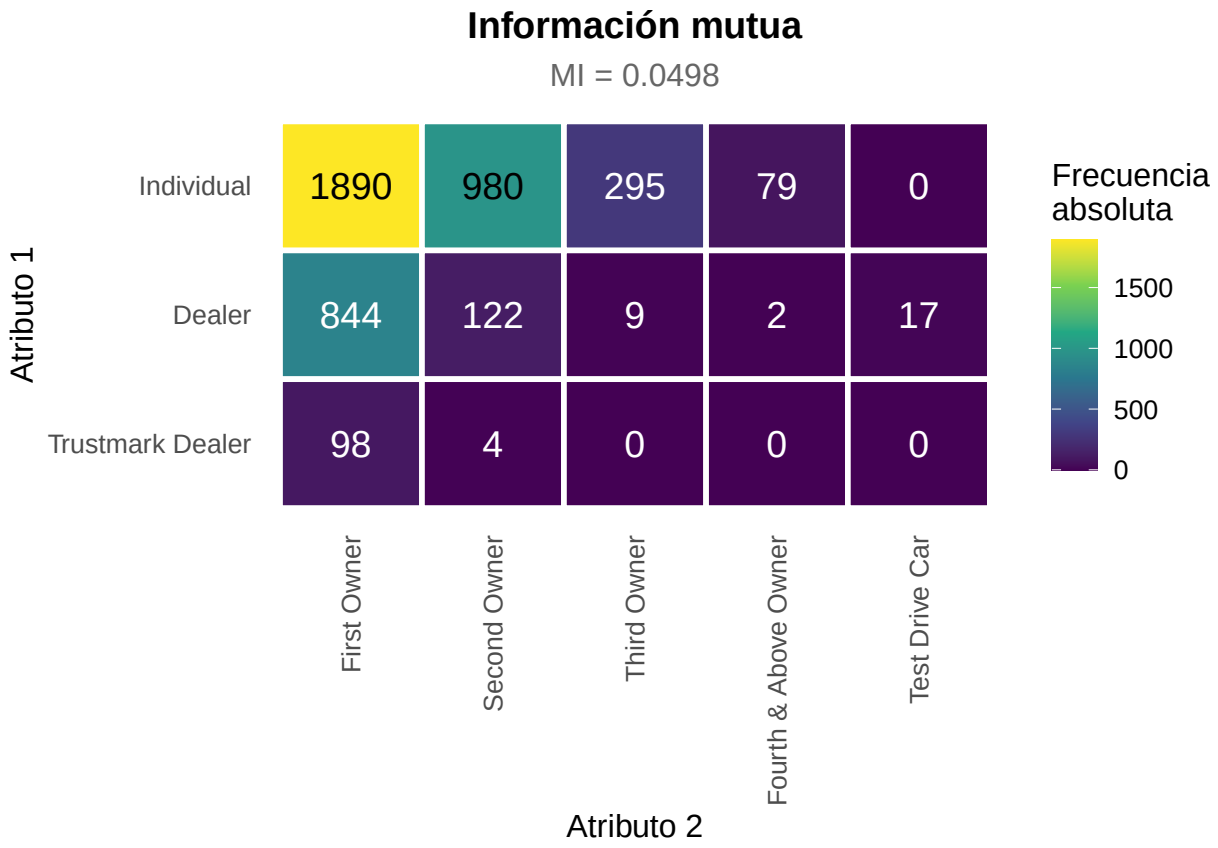
```
## No se ha proporcionado el resultado de información mutua. Se calculará automáticamente.

## Información mutua · Cómo leerla: cada celda cuenta las observaciones de esa combinación; si las filas
## Respuesta: SÍ. la información mutua es positiva y conocer una variable aporta información sobre la o

## Aviso: hay celdas con menos de 5 casos esperados; la prueba G es orientativa.

## Gráfico de información mutua creado correctamente.
```

```
fig
```



Podemos ver como estadisticamente si hay relacion, y a simple vista vemos como los dueños que son individuos, venden coches de segunda, tercera, incluso mas, mientras que los concesionarios no lo hacen.

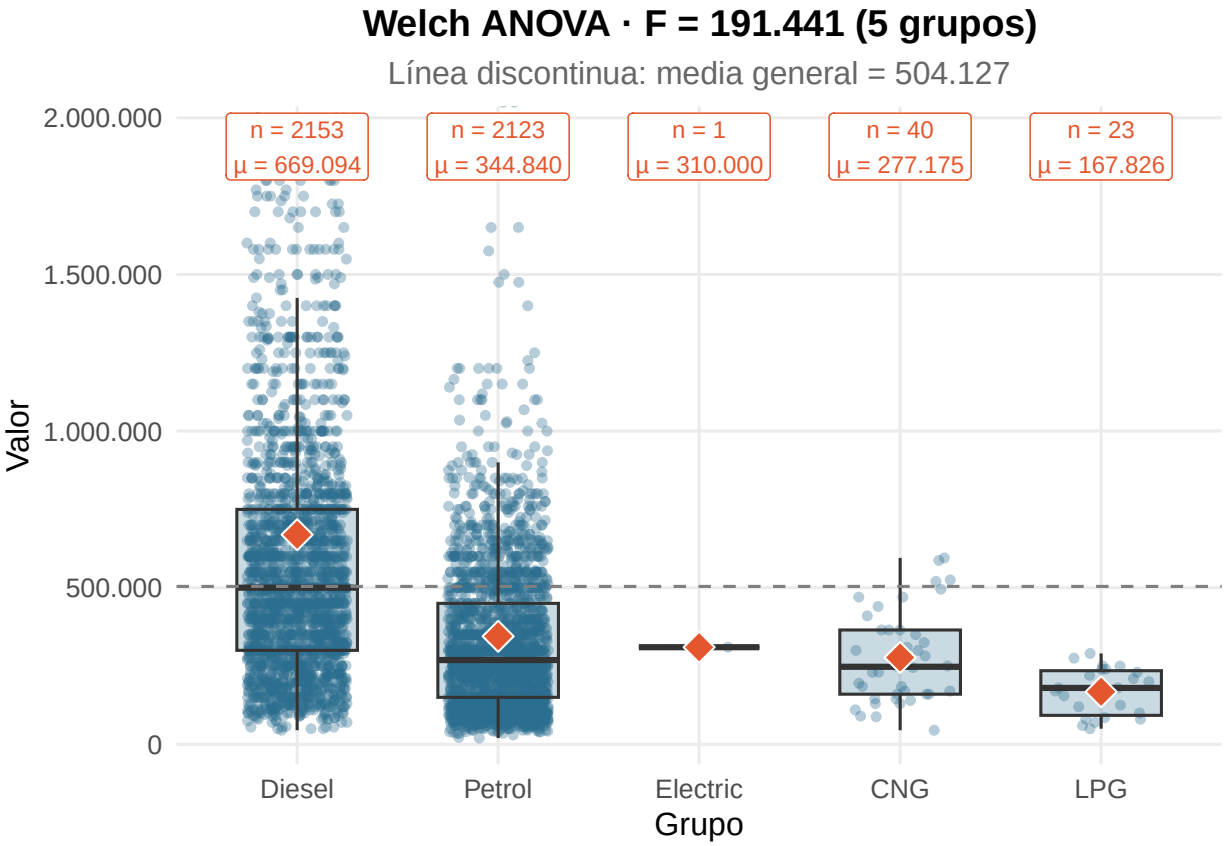
```
fig <- graficar_welch(
  df$fuel,
  df$selling_price,
  limite_y=1700000,
  verbose = 3
)
```

```
## No se ha proporcionado el resultado de Welch ANOVA. Se calculará automáticamente.

## Welch ANOVA · Cómo leerlo: caja = 50% central del grupo, puntos = observaciones, rombo = media del g
## Respuesta: SÍ. el estadístico F es mayor que 1 y las medias muestran diferencias descriptivas entre

## Gráfico welch creado correctamente.
```

```
fig
```



Vemos como de ademas de que caso es mas frecuente el precio de venta medio... varia segun el tipo de fuel del vehiculo.

Conclusión

A lo largo de este documento hemos visto las principales funcionalidades de `SME_R`, desde la preparación de los datos hasta el cálculo de métricas, transformación y selección de variables, análisis de relaciones y visualización de resultados.